

Tyrus 2.1.6 User Guide

Tyrus 2.1.6 User Guide

Table of Contents

Preface	vii
1. Getting Started	1
1.1. WebSocket Services Using Java API for WebSocket	1
1.1.1. Creating Annotated Server Endpoint	1
1.1.2. Client Endpoint	2
1.1.3. Creating Server Endpoint Programmatically	3
1.1.4. Tyrus in Standalone Mode	3
1.2. Creating a New Project from Maven Archetype	4
1.2.1. Executing maven	4
1.2.2. Exploring the Newly Created Project	4
1.2.3. Building the generated project	6
1.2.4. Running the deployed application	7
2. Tyrus Modules and Dependencies	8
3. Deploying WebSocket Endpoints	11
3.1. Deploying Endpoints as a WAR file	11
3.1.1. Deployment Algorithm	11
3.2. Deploying endpoints via <code>jakarta.websocket.server.ServerContainer</code>	12
4. WebSocket API Endpoints, Sessions and MessageHandlers	14
4.1. Endpoint Classes	14
4.1.1. <code>jakarta.websocket.server.ServerEndpoint</code>	14
4.1.2. <code>jakarta.websocket.ClientEndpoint</code>	18
4.2. Endpoint method-level annotations	19
4.2.1. <code>@OnOpen</code>	19
4.2.2. <code>@OnClose</code>	19
4.2.3. <code>@OnError</code>	20
4.2.4. <code>@OnMessage</code>	20
4.3. MessageHandlers	20
5. Configurations	22
5.1. <code>jakarta.websocket.server.ServerEndpointConfig</code>	22
5.2. <code>jakarta.websocket.ClientEndpointConfig</code>	23
6. Endpoint Lifecycle, Sessions, Sending Messages	24
6.1. Endpoint Lifecycle	24
6.2. <code>jakarta.websocket.Session</code>	24
6.3. Sending Messages	25
6.4. RemoteEndpoint	25
6.4.1. <code>jakarta.websocket.RemoteEndpoint.Basic</code>	25
6.4.2. <code>jakarta.websocket.RemoteEndpoint.Async</code>	26
7. Injection Support	27
7.1. <code>jakarta.inject.Inject</code> sample	27
7.2. EJB sample	27
8. Tyrus proprietary configuration	29
8.1. Client-side SSL configuration	29
8.1.1. Host verification	29
8.2. Asynchronous <code>connectToServer</code> methods	30
8.3. Optimized broadcast	31
8.4. Incoming buffer size	31
8.5. Shared client container	32
8.5.1. Custom masking key generator	33
8.6. WebSocket Extensions	33
8.6.1. ExtendedExtension sample	35
8.6.2. Per Message Deflate Extension	37

8.7. Client reconnect	38
8.8. Client behind proxy	39
8.9. JDK 7 client	39
8.9.1. SSL configuration	40
8.10. Tracing support	40
8.10.1. Configuration	40
8.10.2. Tracing Examples	41
8.11. Client handshake request and response logging	43
8.12. JMX Monitoring	43
8.12.1. Configuration	44
8.13. Maximal number of open sessions on server-side	45
8.13.1. Maximal number of open sessions per application	45
8.13.2. Maximal number of open sessions per remote address	46
8.13.3. Maximal number of open sessions per endpoint	46
8.14. Client HTTP Authentication	47
8.14.1. Credentials	47
8.14.2. Auth Configuration	48
8.14.3. User defined authenticator	48
8.14.4. Examples	49
8.15. Client HTTP Redirect	50
8.15.1. Supported HTTP response codes	50
8.15.2. Configuration	50
8.15.3. Exception handling	51
8.16. Client support for HTTP status 503 - Service Unavailable with Retry-After header	51
8.16.1. Configuration	51

List of Tables

2.1. Tyrus core modules	8
2.2. Tyrus containers	9

List of Examples

1.1. Annotated Echo Endpoint	1
1.2. Client Endpoint	2
1.3. Programmatic Echo Endpoint	3
3.1. Deployment of WAR containing several classes extending jakarta.websocket.server.ServerApplicationConfig	11
3.2. Deployment of Annotated Endpoint Using ServerContainer	12
4.1. Echo sample server endpoint.	14
4.2. jakarta.websocket.server.ServerEndpoint with all fields specified	15
4.3. Specifying URI path parameter	15
4.4. SampleDecoder	16
4.5. SampleEncoder	16
4.6. SampleClientEndpoint	18
4.7. @OnOpen with Session and EndpointConfig parameters.	19
4.8. @OnClose with Session and CloseReason parameters.	19
4.9. @OnError with Session and Throwable parameters.	20
4.10. @OnError with Session and Throwable parameters.	20
4.11. MessageHandler basic example	20
5.1. Configuration for EchoEndpoint Deployment	22
5.2. ServerEndpointConfig built using Builder	23
5.3. ClientEndpointConfig built using Builder	23
6.1. Lifecycle echo sample	24
6.2. Sending message in @OnMessage	25
6.3. Sending message via RemoteEndpoint.Basic instance	25
6.4. Method for sending partial text message	26
6.5. Sending message the async way using Future	26
7.1. Injecting bean into jakarta.websocket.server.ServerEndpoint	27
7.2. Echo sample server endpoint.	27

Preface

This is user guide for Tyrus 2.1.6. We are trying to keep it up to date as we add new features. Please use also our API documentation linked from the Tyrus [<https://projects.eclipse.org/projects/ee4j.tyrus>] and Java API for WebSocket [<https://projects.eclipse.org/projects/ee4j.websocket>] home pages as an additional source of information about Tyrus features and API. If you would like to contribute to the guide or have questions on things not covered in our docs, please contact us at tyrus-dev@eclipse.org [<https://dev.eclipse.org/mailman/listinfo/tyrus-dev>].

Chapter 1. Getting Started

This chapter provides a quick introduction on how to get started building WebSocket services using Java API for WebSocket and Tyrus. The example described here presents how to implement simple websocket service as JavaEE web application that can be deployed on any servlet container supporting Servlet 5 and higher. It also discusses starting Tyrus in standalone mode.

1.1. WebSocket Services Using Java API for WebSocket

First, to use the Java API for WebSocket in your project you need to depend on the following artifact:

```
<dependency>
  <groupId>jakarta.websocket</groupId>
  <artifactId>jakarta.websocket-api</artifactId>
  <version>2.1.1</version>
</dependency>
```

1.1.1. Creating Annotated Server Endpoint

In this section we will create a simple server side websocket endpoint which will echo the received message back to the sender. We will deploy this endpoint on the container.

In Java API for WebSocket and Tyrus, there are two basic approaches how to create an endpoint - either annotated endpoint, or programmatic endpoint. By annotated endpoint we mean endpoint constructed by using annotations (`jakarta.websocket.server.ServerEndpoint` for server endpoint and `jakarta.websocket.ClientEndpoint` for client endpoint), like in "Annotated Echo Endpoint".

Example 1.1. Annotated Echo Endpoint

```
1 @ServerEndpoint(value = "/echo")
2 public class EchoEndpointAnnotated {
3     @OnMessage
4     public String onMessage(String message, Session session) {
5         return message;
6     }
7 }
8
```

The functionality of the `EchoEndpointAnnotated` is fairly simple - to send the received message back to the sender. To turn a POJO (Plain Old Java Object) to WebSocket server endpoint, the annotation `@ServerEndpoint(value = "/echo")` needs to be put on the POJO - see line 1. The URI path of the endpoint is `"/echo"`. The annotation `@OnMessage` - line 3 on the method `public String onMessage(String message, Session session)` indicates that this method will be called whenever text message is received. On line 5 in this method the message is sent back to the user by returning it from the message.

The application containing only the `EchoEndpointAnnotated` class can be deployed to the container.

1.1.2. Client Endpoint

Let's create the client part of the application. The client part may be written in JavaScript or any other technology supporting WebSockets. We will use Java API for WebSocket and Tyrus to demonstrate how to develop programmatic client endpoint. The following code is used as a client part to communicate with the `EchoEndpoint` deployed on server using Tyrus and Java API for WebSocket.

The example "Client Endpoint" utilizes the concept of the programmatic endpoint. By programmatic endpoint we mean endpoint which is created by extending class `jakarta.websocket.Endpoint`. The example is standalone java application which needs to depend on some Tyrus artifacts to work correctly, see "Tyrus Standalone Mode". In the example first the `CountDownLatch` is initialized. It is needed as a blocking data structure - on line 31 it either waits for 100 seconds, or until it gets counted down (line 22). On line 9 the `jakarta.websocket.ClientEndpointConfig` is created - we will need it later to connect the endpoint to the server. On line 11 the `org.glassfish.tyrus.client.ClientManager` is created. it implements the `jakarta.websocket.WebSocketContainer` and is used to connect to server. This happens on next line. The client endpoint functionality is contained in the `jakarta.websocket.Endpoint` lazy instantiation. In the `onOpen` method new `MessageHandler` is registered (the received message is just printed on the console and the latch is counted down). After the registration the message is sent to the server (line 25).

Example 1.2. Client Endpoint

```
1 public class DocClient {
2     private static CountDownLatch messageLatch;
3     private static final String SENT_MESSAGE = "Hello World";
4
5     public static void main(String [] args){
6         try {
7             messageLatch = new CountDownLatch(1);
8
9             final ClientEndpointConfig cec = ClientEndpointConfig.Builder.crea
10
11             ClientManager client = ClientManager.createClient();
12             client.connectToServer(new Endpoint() {
13
14                 @Override
15                 public void onOpen(Session session, EndpointConfig config) {
16                     try {
17                         session.addMessageHandler(new MessageHandler.Whole<Str
18
19                         @Override
20                         public void onMessage(String message) {
21                             System.out.println("Received message: "+messag
22                             messageLatch.countDown();
23                         }
24                     });
25                     session.getBasicRemote().sendText(SENT_MESSAGE);
26                 } catch (IOException e) {
27                     e.printStackTrace();
28                 }
29             }
30             }, cec, new URI("ws://localhost:8025/websockets/echo"));
31             messageLatch.await(100, TimeUnit.SECONDS);
```

```
32         } catch (Exception e) {
33             e.printStackTrace();
34         }
35     }
36 }
```

1.1.3. Creating Server Endpoint Programmatically

Similarly to "Client Endpoint" the server registered endpoint may also be the programmatic one:

Example 1.3. Programmatic Echo Endpoint

```
1 public class EchoEndpointProgrammatic extends Endpoint {
2     @Override
3     public void onOpen(final Session session, EndpointConfig config) {
4         session.addMessageHandler(new MessageHandler.Whole<String>() {
5             @Override
6             public void onMessage(String message) {
7                 try {
8                     session.getBasicRemote().sendText(message);
9                 } catch (IOException e) {
10                    e.printStackTrace();
11                }
12            }
13        });
14    }
15 }
```

The functionality of the `EchoEndpointProgrammatic` is fairly simple - to send the received message back to the sender. The programmatic server endpoint needs to extend `jakarta.websocket.Endpoint` - line 1. Method `public void onOpen(final Session session, EndpointConfig config)` gets called once new connection to this endpoint is opened. In this method the `MessageHandler` is registered to the `jakarta.websocket.Session` instance, which opened the connection. Method `public void onMessage(String message)` gets called once the message is received. On line 8 the message is sent back to the sender.

To see how both annotated and programmatic endpoints may be deployed please check the section Deployment. In short: you need to put the server endpoint classes into WAR, deploy on server and the endpoints will be scanned by server and deployed.

1.1.4. Tyrus in Standalone Mode

To use Tyrus in standalone mode it is necessary to depend on correct Tyrus artifacts. The following artifacts need to be added to your pom to use Tyrus:

```
<dependency>
  <groupId>org.glassfish.tyrus</groupId>
  <artifactId>tyrus-server</artifactId>
  <version>2.1.6</version>
</dependency>

<dependency>
```

```
<groupId>org.glassfish.tyrus</groupId>
<artifactId>tyrus-container-grizzly-server</artifactId>
<version>2.1.6</version>
</dependency>
```

Let's use the very same example like for Java API for WebSocket and deploy the `EchoEndpointAnnotated` on the standalone Tyrus server on the hostname "localhost", port 8025 and path "/websockets", so the endpoint will be available at address "ws://localhost:8025/websockets/echo".

```
public void runServer() {
    Server server = new Server("localhost", 8025, "/websockets", null, EchoEndpointAnnotated.class);

    try {
        server.start();
        BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
        System.out.print("Please press a key to stop the server.");
        reader.readLine();
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        server.stop();
    }
}
```

1.2. Creating a New Project from Maven Archetype

1.2.1. Executing maven

Since starting from a Maven project is the most convenient way for working with Tyrus, let's now have a look at this approach. We will now create a new Tyrus project that runs on top of a Grizzly [<https://eclipse-ee4j.github.io/grizzly/>] container. We will use a Tyrus-provided maven archetype. To create the project, execute the following Maven command in the directory where the new project should reside:

```
mvn archetype:generate -DarchetypeArtifactId=tyrus-archetype-echo \
-DarchetypeGroupId=org.glassfish.tyrus.archetypes -DinteractiveMode=false \
-DgroupId=com.example -DartifactId=echo-service -Dpackage=com.example \
-DarchetypeVersion=2.1.6
```

Feel free to adjust the `groupId`, `package` and/or `artifactId` of your new project. Alternatively, you can change it by updating the new project `pom.xml` once it gets generated.

1.2.2. Exploring the Newly Created Project

Once the project generation from a Tyrus maven archetype is successfully finished, you should see the new `echo-service` project directory created in your current location. The directory contains a standard Maven project structure:

Project build and management configuration is described in the `pom.xml` located in the project root directory.

Project sources are located under `src/main/java`.

Project resources are located under `src/main/resources`.

Project web application files are located under `src/main/webapp`.

The project contains the simple `EchoEndpoint` class which looks (similar to "Annotated Echo Endpoint") as follows:

```
1 package com.example;
2
3 import jakarta.websocket.OnMessage;
4 import jakarta.websocket.Session;
5 import jakarta.websocket.server.ServerEndpoint;
6
7 /**
8  * Server endpoint "listening" on path "/echo".
9  */
10 @ServerEndpoint("/echo")
11 public class EchoEndpoint {
12
13     /**
14      * Method handling incoming text messages.
15      *
16      * This implementations just sends back received message.
17      *
18      * @param message received message.
19      * @return returned value will be sent back to client. You can also declare
20      * that case, you would need to obtain {@link jakarta.websocket.Session} ob
21      * {@link jakarta.websocket.Session#getBasicRemote()}#sendText();} in order
22      */
23     @OnMessage
24     public String echo(String message) {
25         return message;
26     }
27 }
```

Another piece of code that has been generated in this skeleton project is a `EchoTest` unit test class that is located in the same `com.example` package as the `EchoEndpoint` class, however, this unit test class is placed into the maven project test source directory `src/test/java` (similar to "Client Endpoint"):

```
1 public class EchoTest extends TestContainer {
2
3     public static final String MESSAGE = "Do or do not, there is no try.";
4
5     @Test
6     public void testEcho() throws DeploymentException, InterruptedException, I
7         final Server server = startServer(EchoEndpoint.class);
8
9         final CountDownLatch messageLatch = new CountDownLatch(1);
10
11     try {
12         final ClientManager client = ClientManager.createClient();
13         client.connectToServer(new Endpoint() {
14             @Override
15             public void onOpen(Session session, EndpointConfig EndpointCon
16
17             try {
```

```
18         session.addMessageHandler(new MessageHandler.Whole<String>() {
19             @Override
20             public void onMessage(String message) {
21                 System.out.println("# Received: " + message);
22
23                 if (message.equals(MESSAGE)) {
24                     messageLatch.countDown();
25                 }
26             }
27         });
28
29         session.getBasicRemote().sendText(MESSAGE);
30     } catch (IOException e) {
31         // do nothing
32     }
33 }
34 }, ClientEndpointConfig.Builder.create().build(), getURI(EchoEndpoint));
35
36 assertTrue(messageLatch.await(1, TimeUnit.SECONDS));
37
38 } catch (Exception e) {
39     e.printStackTrace();
40     fail();
41 } finally {
42     stopServer(server);
43 }
44 }
45 }
```

In this unit test, a Grizzly container is first started and server application is deployed using the static call `TestContainer.startServer()`. Next, WebSocket client instance `client` is built and then a programmatic WebSocket Endpoint is used to connect to the `EchoEndpoint` deployed on the Grizzly server at `ws://localhost:8025/e2e-test/echo`. The value of port 8025 and context path `e2e-test` are default values preset at the `TestContainer`.

The generated project structure also contains an `index.html` file, stored in `src/test/webapp` folder. This web page contains a javascript WebSocket client that is capable of sending WebSocket requests to the `EchoEndpoint`. This web page is deployed along the war application in the WebSocket container such as Tyrus.

1.2.3. Building the generated project

Now that we have seen the content of the project, let's try to test-run it. To do this, we need to invoke following command on the command line:

```
mvn clean package
```

This will compile the project and run the project unit tests. We should see a similar output that informs about a successful build once the build is finished:

Results :

```
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

```
[INFO] Packaging webapp
[INFO] Assembling webapp [echo-service] in [.../echo-service/target/echo-service]
[INFO] Processing war project
[INFO] Copying webapp resources [.../echo-service/src/main/webapp]
[INFO] Building war: .../echo-service/target/echo-service.war
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.979 s
[INFO] Finished at: 2022-01-28T14:23:45+01:00
[INFO] Final Memory: 17M/490M
[INFO] -----
```

Now you are ready to take the packaged WAR (located under `./target/echo-service.war`) and deploy it to a WebSocket container of your choice.

Important

To deploy the application, you will need a WebSocket compliant container (such as Tyrus). Tyrus can be found for instance in Glassfish 7. It is also possible to create a packaged WAR with Tyrus jars included to deploy to a Servlet 5/Servlet 6 compliant container.

1.2.4. Running the deployed application

Once the war is deployed, it can be run on the deployment URI such as `localhost:8080/echo-service/` (the included `index.html` web page can be opened in a web browser). The web page has a title `Web Socket Echo Client` and contains a button with a label `Press me` and a text field prefilled with `Hello Web Sockets !`. When the button is pressed, the WebSocket echo service queried and the web page contains a communication:

CONNECTED

SENT: Hello Web Sockets !

RECEIVED: Hello Web Sockets !

Chapter 2. Tyrus Modules and Dependencies

Tyrus is built, assembled and installed using Maven. Tyrus is deployed to maven.org maven repository at the following location:<http://search.maven.org/>. Jars, jar sources, jar javadoc and samples are all available on the java.net maven repository.

All Tyrus components are built using Java SE 7 compiler. It means, you will also need at least Java SE 7 to be able to compile and run your application. Developers using maven are likely to find it easier to include and manage dependencies of their applications than developers using ant or other build technologies. The following table provides an overview of all Tyrus modules and their dependencies with links to the respective binaries.

Table 2.1. Tyrus core modules

Module	Dependencies	Description
tyrus-server [http://search.maven.org/#artifactdetails org.glassfish.tyrus tyrus-server 2.1.6 jar]	tyrus-core, tyrus-spi, tyrus-websocket-core	Basic server functionality
tyrus-core [http://search.maven.org/#artifactdetails org.glassfish.tyrus tyrus-core 2.1.6 jar]	tyrus-spi, tyrus-websocket-core	Core Tyrus functionality
tyrus-client [http://search.maven.org/#artifactdetails org.glassfish.tyrus tyrus-client 2.1.6 jar]	tyrus-core, tyrus-spi, tyrus-websocket-core	Basic client functionality
tyrus-documentation [http://search.maven.org/#artifactdetails org.glassfish.tyrus tyrus-documentation 2.1.6 jar]	[nothing]	Project documentation
tyrus-websocket-core [http://search.maven.org/#artifactdetails org.glassfish.tyrus tyrus-websocket-core 2.1.6 jar]	[nothing]	The WebSocket protocol

Module	Dependencies	Description
search.maven.org/ #artifactdetails org.glassfish.tyrus tyrus- websocket- core 2.1.6 jar]		
tyrus- samples [http:// search.maven.org/ remotecontent? filepath=org/ glassfish/ tyrus/ bundles/ tyrus- samples/2.1.6/ tyrus- samples-2.1.6- all.zip]	tyrus-server, tyrus-client, tyrus- container-grizzly, tyrus-core, tyrus- spi, tyrus-websocket-core	Samples of using Java API for WebSocket and Tyrus
tyrus-spi [http:// search.maven.org/ #artifactdetails org.glassfish.tyrus tyrus-spi 2.1.6 jar]	[nothing]	SPI

Table 2.2. Tyrus containers

Module	Dependencies	Description
tyrus- container- glassfish- cdi [http:// search.maven.org/ #artifactdetails org.glassfish.tyrus tyrus- container- glassfish-cdi 2.1.6 jar]	tyrus-spi	CDI support
tyrus- container- glassfish- ejb [http:// search.maven.org/ #artifactdetails org.glassfish.tyrus tyrus-	tyrus-spi	EJB support

Module	Dependencies	Description
container-glassfish-ejb 2.1.6 jar]		
tyrus-container-grizzly [http://search.maven.org/#artifactdetails org.glassfish.tyrus tyrus-container-grizzly 2.1.6 jar]	tyrus-core, tyrus-spi, tyrus-websocket-core	Grizzly integration for Tyrus client and standalone server usage
tyrus-container-servlet [http://search.maven.org/#artifactdetails org.glassfish.tyrus tyrus-container-servlet 2.1.6 bundle]	tyrus-server, tyrus-core, tyrus-spi, tyrus-websocket-core	Servlet support for integration into web containers

Chapter 3. Deploying WebSocket Endpoints

Deploying WebSocket endpoints can be done in two ways. Either deploying via putting the endpoint in the WAR file, or using the `ServerContainer` methods to deploy the programmatic endpoint in the deployment phase.

3.1. Deploying Endpoints as a WAR file

The classes that are scanned for in WAR are the following ones:

- Classes that implement the `jakarta.websocket.ServerApplicationConfig`.
- Classes annotated with `jakarta.websocket.server.ServerEndpoint`.
- Classes that extend `jakarta.websocket.Endpoint`.

3.1.1. Deployment Algorithm

1. If one or more classes implementing `ServerApplicationConfiguration` are present in the WAR file, Tyrus deploys endpoints provided by all of these classes. Tyrus doesn't deploy any other classes present in the WAR (annotated by `jakarta.websocket.server.ServerEndpoint` or extending `jakarta.websocket.Endpoint`).
2. If no class implementing `ServerApplicationConfiguration` is present, Tyrus deploys all classes annotated with `@ServerEndpoint` or extending `Endpoint` present in the WAR.

Let's have the following classes in the WAR:

Example 3.1. Deployment of WAR containing several classes extending `jakarta.websocket.server.ServerApplicationConfig`

```
1 public class MyApplicationConfigOne implements ServerApplicationConfig {
2     public Set<ServerEndpointConfig> getEndpointConfigs(Set<Class<? extends Endpoint>> scanned);
3     Set<Class<? extends Endpoint>> s = new HashSet<Class<? extends Endpoint>>();
4     s.add(ProgrammaticEndpointOne.class);
5     return s;
6 }
7
8 public Set<Class> getAnnotatedEndpointClasses(Set<Class<?>> scanned);
9     Set<Class<?>> s = new HashSet<Class<?>>();
10    s.add(AnnotatedEndpointOne.class);
11    return s;
12 }
13 }
14
15 public class MyApplicationConfigTwo implements ServerApplicationConfig {
16     public Set<ServerEndpointConfig> getEndpointConfigs(Set<Class<? extends Endpoint>> scanned);
17     Set<Class<? extends Endpoint>> s = new HashSet<Class<? extends Endpoint>>();
18     s.add(ProgrammaticEndpointTwo.class);
19     return s;
20 }
```

```
21
22 public Set<Class> getAnnotatedEndpointClasses(Set<Class<?>> scanned);
23     Set<Class<?>> s = new HashSet<Class<?>>();
24     s.add(AnnotatedEndpointTwo.class);
25     return s;
26 }
27 }
28
29 @ServerEndpoint(value = "/annotatedone")
30 public class AnnotatedEndpointOne {
31     ...
32 }
33
34 @ServerEndpoint(value = "/annotatedtwo")
35     public class AnnotatedEndpointTwo {
36     ...
37 }
38
39 @ServerEndpoint(value = "/annotatedthree")
40 public class AnnotatedEndpointThree {
41     ...
42 }
43
44 public class ProgrammaticEndpointOne extends Endpoint {
45     ...
46 }
47
48 public class ProgrammaticEndpointTwo extends Endpoint {
49     ...
50 }
51
52 public class ProgrammaticEndpointThree extends Endpoint {
53     ...
54 }
```

According to the deployment algorithm classes `AnnotatedEndpointOne`, `AnnotatedEndpointTwo`, `ProgrammaticEndpointOne` and `ProgrammaticEndpointTwo` will be deployed. `AnnotatedEndpointThree` and `ProgrammaticEndpointThree` will not be deployed, as these are not returned by the respective methods of `MyApplicationConfigOne` nor `MyApplicationConfigTwo`.

3.2. Deploying endpoints via `jakarta.websocket.server.ServerContainer`

Endpoints may be deployed using `jakarta.websocket.server.ServerContainer` during the application initialization phase. For websocket enabled web containers, developers may obtain a reference to the `ServerContainer` instance by retrieving it as an attribute named `jakarta.websocket.server.ServerContainer` on the `ServletContext`, see the following example for annotated endpoint:

Example 3.2. Deployment of Annotated Endpoint Using `ServerContainer`

```
1 @WebListener
```

```
2 @ServerEndpoint("/annotated")
3 public class MyServletContextListenerAnnotated implements ServletContextListen
4
5     @Override
6     public void contextInitialized(ServletContextEvent servletContextEvent) {
7         final ServerContainer serverContainer = (ServerContainer) servletConte
8                                     .getAttribute("jakarta.web
9
10        try {
11            serverContainer.addEndpoint(MyServletContextListenerAnnotated.clas
12        } catch (DeploymentException e) {
13            e.printStackTrace();
14        }
15    }
16
17    @OnMessage
18    public String onMessage(String message) {
19        return message;
20    }
21
22    @Override
23    public void contextDestroyed(ServletContextEvent servletContextEvent) {
24    }
25 }
```

Chapter 4. WebSocket API Endpoints, Sessions and MessageHandlers

This chapter presents an overview of the core WebSocket API concepts - endpoints, configurations and message handlers.

The JAVA API for WebSocket specification draft can be found online here [<http://jcp.org/aboutJava/communityprocess/pfd/jsr356/index.html>].

4.1. Endpoint Classes

Server endpoint classes are POJOs (Plain Old Java Objects) that are annotated with `jakarta.websocket.server.ServerEndpoint`. Similarly, *client endpoint classes* are POJOs annotated with `jakarta.websocket.ClientEndpoint`. This section shows how to use Tyrus to annotate Java objects to create WebSocket web services.

The following code example is a simple example of a WebSocket endpoint using annotations. The example code shown here is from echo sample which ships with Tyrus.

Example 4.1. Echo sample server endpoint.

```
1 @ServerEndpoint("/echo")
2 public class EchoEndpoint {
3
4     @OnOpen
5     public void onOpen(Session session) throws IOException {
6         session.getBasicRemote().sendText("onOpen");
7     }
8
9     @OnMessage
10    public String echo(String message) {
11        return message + " (from your server)";
12    }
13
14    @OnError
15    public void onError(Throwable t) {
16        t.printStackTrace();
17    }
18
19    @OnClose
20    public void onClose(Session session) {
21
22    }
23 }
```

Let's explain the JAVA API for WebSocket annotations.

4.1.1. `jakarta.websocket.server.ServerEndpoint`

`jakarta.websocket.server.ServerEndpoint` has got one mandatory field - *value* and four optional fields. See the example below.

Example 4.2. jakarta.websocket.server.ServerEndpoint with all fields specified

```
1 @ServerEndpoint(  
2     value = "/sample",  
3     decoders = ChatDecoder.class,  
4     encoders = DisconnectResponseEncoder.class,  
5     subprotocols = {"subprtocol1", "subprotocol2"},  
6     configurator = Configurator.class  
7 )  
8 public class SampleEndpoint {  
9  
10     @OnMessage  
11     public SampleResponse receiveMessage(SampleType message, Session session)  
12     {  
13         return new SampleResponse(message);  
14     }  
15 }
```

4.1.1.1. value

Denotes a relative URI path at which the server endpoint will be deployed. In the example "jakarta.websocket.server.ServerEndpoint with all fields specified", the Java class will be hosted at the URI path /sample. The field *value* must begin with a '/' and may or may not end in a '/', it makes no difference. Thus request URLs that end or do not end in a '/' will both be matched. WebSocket API for JAVA supports level 1 URI templates.

URI path templates are URIs with variables embedded within the URI syntax. These variables are substituted at runtime in order for a resource to respond to a request based on the substituted URI. Variables are denoted by curly braces. For example, look at the following @ServerEndpoint annotation:

```
@ServerEndpoint("/users/{username}")
```

In this type of example, a user will be prompted to enter their name, and then a Tyrus web service configured to respond to requests to this URI path template will respond. For example, if the user entered their username as "Galileo", the web service will respond to the following URL: `http://example.com/users/Galileo`

To obtain the value of the username variable the `jakarta.websocket.server.PathParam` may be used on method parameter of methods annotated with one of `@OnOpen`, `@OnMessage`, `@OnError`, `@OnClose`.

Example 4.3. Specifying URI path parameter

```
1 @ServerEndpoint("/users/{username}")  
2 public class UserEndpoint {  
3  
4     @OnMessage  
5     public String getUser(String message, @PathParam("username") String userNa  
6     ...  
7 }  
8 }
```

4.1.1.2. decoders

Contains list of classes that will be used to decode incoming messages for the endpoint. By decoding we mean transforming from text / binary websocket message to some user defined type. Each decoder needs to implement the Decoder interface.

`SampleDecoder` in the following example decodes String message and produces `SampleType` message - see decode method on line 4.

Example 4.4. SampleDecoder

```
1 public class SampleDecoder implements Decoder.Text<SampleType> {
2
3     @Override
4     public SampleType decode(String s) {
5         return new SampleType(s);
6     }
7
8     @Override
9     public boolean willDecode(String s) {
10        return s.startsWith(SampleType.PREFIX);
11    }
12
13    @Override
14    public void init(EndpointConfig config) {
15        // do nothing.
16    }
17
18    @Override
19    public void destroy() {
20        // do nothing.
21    }
22 }
```

4.1.1.3. encoders

Contains list of classes that will be used to encode outgoing messages. By encoding we mean transforming message from user defined type to text or binary type. Each encoder needs to implement the `Encoder` interface.

`SampleEncoder` in the following example decodes String message and produces `SampleType` message - see decode method on line 4.

Example 4.5. SampleEncoder

```
1 public class SampleEncoder implements Encoder.Text<SampleType> {
2
3     @Override
4     public String encode(SampleType message) {
5         return data.toString();
6     }
7
8     @Override
9     public void init(EndpointConfig config) {
10        // do nothing.
11    }
12
13    @Override
14    public void destroy() {
15        // do nothing.
16    }
17 }
```

```
16     }  
17 }
```

4.1.1.4. subprotocols

List of names (Strings) of supported sub-protocols. The first protocol in this list that matches with sub-protocols provided by the client side is used.

4.1.1.5. configurator

Users may provide their own implementation of `ServerEndpointConfiguration.Configurator`. It allows them to control some algorithms used by Tyrus in the connection initialization phase:

- `public String getNegotiatedSubprotocol(List<String> supported, List<String> requested)` allows the user to provide their own algorithm for selection of used subprotocol.
- `public List<Extension> getNegotiatedExtensions(List<Extension> installed, List<Extension> requested)` allows the user to provide their own algorithm for selection of used Extensions.
- `public boolean checkOrigin(String originHeaderValue)`. allows the user to specify the origin checking algorithm.
- `public void modifyHandshake(ServerEndpointConfig sec, HandshakeRequest request, HandshakeResponse response)` . allows the user to modify the handshake response that will be sent back to the client.
- `public <T> T getEndpointInstance(Class<T> endpointClass) throws InstantiationException` . allows the user to provide the way how the instance of an Endpoint is created

```
1 public class ConfiguratorTest extends ServerEndpointConfig.Configurator{  
2  
3     public String getNegotiatedSubprotocol(List<String> supported, List<String>  
4         // Plug your own algorithm here  
5     }  
6  
7     public List<Extension> getNegotiatedExtensions(List<Extension> installed,  
8         // Plug your own algorithm here  
9     }  
10  
11     public boolean checkOrigin(String originHeaderValue) {  
12         // Plug your own algorithm here  
13     }  
14  
15     public void modifyHandshake(ServerEndpointConfig sec, HandshakeRequest req  
16         // Plug your own algorithm here  
17     }  
18  
19     public <T> T getEndpointInstance(Class<T> endpointClass) throws Instantiat  
20         // Plug your own algorithm here  
21     }  
22 }
```


4.1.2. jakarta.websocket.ClientEndpoint

The `@ClientEndpoint` class-level annotation is used to turn a POJO into WebSocket client endpoint. In the following sample the client sends text message "Hello!" and prints out each received message.

Example 4.6. SampleClientEndpoint

```
1 @ClientEndpoint(  
2     decoders = SampleDecoder.class,  
3     encoders = SampleEncoder.class,  
4     subprotocols = {"subprtocol1", "subprotocol2"},  
5     configurator = ClientConfigurator.class)  
6 public class SampleClientEndpoint {  
7  
8     @OnOpen  
9     public void onOpen(Session p) {  
10         try {  
11             p.getBasicRemote().sendText("Hello!");  
12         } catch (IOException e) {  
13             e.printStackTrace();  
14         }  
15     }  
16  
17     @OnMessage  
18     public void onMessage(String message) {  
19         System.out.println(String.format("%s %s", "Received message: ", message));  
20     }  
21 }  
22
```

4.1.2.1. decoders

Contains list of classes that will be used to decode incoming messages for the endpoint. By decoding we mean transforming from text / binary websocket message to some user defined type. Each decoder needs to implement the `Decoder` interface.

4.1.2.2. encoders

Contains list of classes that will be used to encode outgoing messages. By encoding we mean transforming message from user defined type to text or binary type. Each encoder needs to implement the `Encoder` interface.

4.1.2.3. subprotocols

List of names (Strings) of supported sub-protocols.

4.1.2.4. configurator

Users may provide their own implementation of `ClientEndpointConfiguration.Configurator`. It allows them to control some algorithms used by Tyrus in the connection initialization phase. Method *beforeRequest* allows the user to change the request headers constructed by Tyrus. Method *afterResponse* allows the user to process the handshake response.

```
1 public class Configurator {
2
3     public void beforeRequest(Map<String, List<String>> headers) {
4         //affect the headers before request is sent
5     }
6
7     public void afterResponse(HandshakeResponse hr) {
8         //process the handshake response
9     }
10 }
```

4.2. Endpoint method-level annotations

4.2.1. @OnOpen

This annotation may be used on certain methods of `@ServerEndpoint` or `@ClientEndpoint`, but only once per endpoint. It is used to decorate a method which is called once new connection is established. The connection is represented by the optional `Session` parameter. The other optional parameter is `EndpointConfig`, which represents the passed configuration object. Note that the `EndpointConfig` allows the user to access the user properties.

Example 4.7. @OnOpen with Session and EndpointConfig parameters.

```
1 @ServerEndpoint("/sample")
2 public class EchoEndpoint {
3
4     private Map<String, Object> properties;
5
6     @OnOpen
7     public void onOpen(Session session, EndpointConfig config) throws IOException {
8         session.getBasicRemote().sendText("onOpen");
9         properties = config.getUserProperties();
10    }
11 }
```

4.2.2. @OnClose

This annotation may be used on any method of `@ServerEndpoint` or `@ClientEndpoint`, but only once per endpoint. It is used to decorate a method which is called once the connection is being closed. The method may have one `Session` parameter, one `CloseReason` parameter and parameters annotated with `@PathParam`.

Example 4.8. @OnClose with Session and CloseReason parameters.

```
1 @ServerEndpoint("/sample")
2 public class EchoEndpoint {
3
4     @OnClose
5     public void onClose(Session session, CloseReason reason) throws IOException {
6         //prepare the endpoint for closing.
7     }
8 }
```

4.2.3. @OnError

This annotation may be used on any method of `@ServerEndpoint` or `@ClientEndpoint`, but only once per endpoint. It is used to decorate a method which is called once Exception is being thrown by any method annotated with `@OnOpen`, `@OnMessage` and `@OnClose`. The method may have optional Session parameter and Throwable parameters.

Example 4.9. @OnError with Session and Throwable parameters.

```
1 @ServerEndpoint("/sample")
2 public class EchoEndpoint {
3
4     @OnError
5     public void onError(Session session, Throwable t) {
6         t.printStackTrace();
7     }
8 }
```

4.2.4. @OnMessage

This annotation may be used on certain methods of `@ServerEndpoint` or `@ClientEndpoint`, but only once per endpoint. It is used to decorate a method which is called once new message is received.

Example 4.10. @OnMessage with Session and Throwable parameters.

```
1 @ServerEndpoint("/sample")
2 public class EchoEndpoint {
3
4     @OnMessage
5     public void onMessage(Session session, String message) {
6         System.out.println("Received message: " + message);
7     }
8 }
```

4.3. MessageHandlers

Implementing the `jakarta.websocket.MessageHandler` interface is one of the ways how to receive messages on endpoints (both server and client). It is aimed primarily on programmatic endpoints, as the annotated ones use the method level annotation `jakarta.websocket.OnMessage` to denote the method which receives messages.

The MessageHandlers get registered on the Session instance:

Example 4.11. MessageHandler basic example

```
1 public class MyEndpoint extends Endpoint {
2
3     @Override
4     public void onOpen(Session session, EndpointConfig EndpointConfig) {
5         session.addMessageHandler(new MessageHandler.Whole<String>() {
6             @Override
7             public void onMessage(String message) {
```

```
8         System.out.println("Received message: "+message);
9     }
10    });
11 }
12 }
```

There are two orthogonal criterions which classify MessageHandlers. According the WebSocket Protocol (RFC 6455) the message may be sent either complete, or in chunks. In Java API for WebSocket this fact is reflected by the interface which the handler implements. Whole messages are processed by handler which implements `jakarta.websocket.MessageHandler.Whole` interface. Partial messages are processed by handlers that implement `jakarta.websocket.MessageHandler.Partial` interface. However, if user registers just the whole message handler, it doesn't mean that the handler will process solely whole messages. If partial message is received, the parts are cached by Tyrus until the final part is received. Then the whole message is passed to the handler. Similarly, if the user registers just the partial message handler and whole message is received, it is passed directly to the handler.

The second criterion is the data type of the message. WebSocket Protocol (RFC 6455) defines four message data type - text message, According to Java API for WebSocket the text messages will be processed by MessageHandlers with the following types:

- `java.lang.String`
- `java.io.Reader`
- any developer object for which there is a corresponding `jakarta.websocket.Decoder.Text` or `jakarta.websocket.Decoder.TextStream`.

The binary messages will be processed by MessageHandlers with the following types:

- `java.nio.ByteBuffer`
- `java.io.InputStream`
- any developer object for which there is a corresponding `jakarta.websocket.Decoder.Binary` or `jakarta.websocket.Decoder.BinaryStream`.

The Java API for WebSocket limits the registration of MessageHandlers per Session to be one MessageHandler per native websocket message type. In other words, the developer can only register at most one MessageHandler for incoming text messages, one MessageHandler for incoming binary messages, and one MessageHandler for incoming pong messages. This rule holds for both whole and partial message handlers, i.e there may be one text MessageHandler - either whole, or partial, not both.

Chapter 5. Configurations

`jakarta.websocket.server.ServerEndpointConfig` and `jakarta.websocket.ClientEndpointConfig` objects are used to provide the user the ability to configure websocket endpoints. Both server and client endpoints have some part of configuration in common, namely encoders, decoders, and user properties. The user properties may developers use to store the application specific data. For the developer's convenience the builders are provided for both `ServerEndpointConfig` and `ClientEndpointConfig`.

5.1. `jakarta.websocket.server.ServerEndpointC`

The `jakarta.websocket.server.ServerEndpointConfig` is used when deploying the endpoint either via implementing the `jakarta.websocket.server.ServerApplicationConfig`, or via registering the programmatic endpoint at the `jakarta.websocket.server.ServerContainer` instance. It allows the user to create the configuration programmatically.

The following example is used to deploy the `EchoEndpoint` programmatically. In the method `getEndpointClass()` the user has to specify the class of the deployed endpoint. In the example `Tyrus` will create an instance of `EchoEndpoint` and deploy it. This is the way how to tie together endpoint and its configuration. In the method `getPath()` the user specifies that the endpoint instance will be deployed at the path `"/echo"`. In the method `public List<String> getSubprotocols()` the user specifies that the supported subprotocols are `"echo1"` and `"echo2"`. The method `getExtensions()` defines the extensions the endpoint supports. Similarly the example configuration does not use any configurator. Method `public List<Class<? extends Encoder>> getEncoders()` defines the encoders used by the endpoint. The decoders and user properties map are defined in similar fashion.

If the endpoint class which is about to be deployed is an annotated endpoint, note that the endpoint configuration will be taken from configuration object, not from the annotation on the endpoint class.

Example 5.1. Configuration for `EchoEndpoint` Deployment

```
1 public class EchoEndpointConfig implements ServerEndpointConfig{
2
3     private final Map<String, Object> userProperties = new HashMap<String, Obj
4
5     @Override
6     public Class<?> getEndpointClass() {
7         return EchoEndpoint.class;
8     }
9
10    @Override
11    public String getPath() {
12        return "/echo";
13    }
14
15    @Override
16    public List<String> getSubprotocols() {
17        return Arrays.asList("echo1", "echo2");
18    }
19
20    @Override
```

```
21     public List<Extension> getExtensions() {
22         return null;
23     }
24
25     @Override
26     public Configurator getConfigurator() {
27         return null;
28     }
29
30     @Override
31     public List<Class<? extends Encoder>> getEncoders() {
32         return Arrays.asList(SampleEncoder.class);
33     }
34
35     @Override
36     public List<Class<? extends Decoder>> getDecoders() {
37         return Arrays.asList(SampleDecoder.class);
38     }
39
40     @Override
41     public Map<String, Object> getUserProperties() {
42         return userProperties;
43     }
44 }
```

To make the development easy the *jakarta.websocket.server.ServerEndpointConfig* provides a builder to construct the configuration object:

Example 5.2. ServerEndpointConfig built using Builder

```
1 ServerEndpointConfig config = ServerEndpointConfig.Builder.create(EchoEndpoint
2     decoders(Arrays.<Class<? extends Decoder>>asList(JsonDecoder.class)).
3     encoders(Arrays.<Class< extends Encoder>>asList(JsonEncoder.class)).build(
```

5.2. jakarta.websocket.ClientEndpointConfig

The *jakarta.websocket.ClientEndpointConfig* is used when deploying the programmatic client endpoint via registering the programmatic endpoint at the *WebSocketContainer* instance. Some of the configuration methods come from the *EndpointConfig* class, which is extended by both *jakarta.websocket.server.ServerEndpointConfig* and *jakarta.websocket.ClientEndpointConfig*. Then there are methods for configuring the preferred subprotocols the client endpoint wants to use and supported extensions. It is also possible to use the *ClientEndpointConfig.Configurator* in order to be able to affect the endpoint behaviour before and after request.

Similarly to the *ServerEndpointConfig*, there is a *Builder* provided to construct the configuration easily:

Example 5.3. ClientEndpointConfig built using Builder

```
1 ClientEndpointConfig.Builder.create().
2     decoders(Arrays.<Class<? extends Decoder>>asList(JsonDecoder.class)).
3     encoders(Arrays.<Class<? extends Encoder>>asList(JsonEncoder.class)).
4     preferredSubprotocols(Arrays.asList("echo1", "echo2")).build();
```

Chapter 6. Endpoint Lifecycle, Sessions, Sending Messages

6.1. Endpoint Lifecycle

As mentioned before, the endpoint in Java API for WebSocket is represented either by instance of `jakarta.websocket.Endpoint`, or by class annotated with either `jakarta.websocket.server.ServerEndpoint` or `jakarta.websocket.ClientEndpoint`. Unless otherwise defined by developer provided configurator (defined in instance of `jakarta.websocket.server.ServerEndpointConfig` or `jakarta.websocket.ClientEndpointConfig`, Tyrus uses one endpoint instance per VM per connected peer. Therefore one endpoint instance typically handles connections from one peer.

6.2. `jakarta.websocket.Session`

The sequence of interactions between an endpoint instance and remote peer is in Java API for WebSocket modelled by `jakarta.websocket.Session` instance. This interaction starts by mandatory open notification, continues by 0 - n websocket messages and is finished by mandatory closing notification.

The `jakarta.websocket.Session` instance is passed by Tyrus to the user in the following methods for programmatic endpoints:

- `public void onOpen(Session session, EndpointConfig config)`
- `public void onClose(Session session, CloseReason closeReason)`
- `public void onError(Session session, Throwable thr)`

The `jakarta.websocket.Session` instance is passed by Tyrus to the user in the methods annotated by following annotations for annotated endpoints:

- method annotated with `jakarta.websocket.OnOpen`
- method annotated with `jakarta.websocket.OnMessage`
- method annotated with `jakarta.websocket.OnClose`
- method annotated with `jakarta.websocket.OnError`

In each of the methods annotated with the preceeding annotations the user may use parameter of type `jakarta.websocket.Session`. In the following example the developer wants to send a message in the method annotated with `jakarta.websocket.OnOpen`. As we will demonstrate later, the developer needs the session instance to do so. According to Java API for WebSocket Session is one of the allowed parameters in methods annotated with `jakarta.websocket.OnOpen`. Once the annotated method gets called, Tyrus passes in the correct instance of `jakarta.websocket.Session`.

Example 6.1. Lifecycle echo sample

```
1 @ServerEndpoint("/echo")
2 public class EchoEndpoint {
3
```

```
4    @OnOpen
5    public void onOpen(Session session) throws IOException {
6        session.getBasicRemote().sendText("onOpen");
7    }
8
9    @OnMessage
10   public String echo(String message) {
11       return message;
12   }
13
14   @OnError
15   public void onError(Throwable t) {
16       t.printStackTrace();
17   }
18 }
```

6.3. Sending Messages

Generally there are two ways how to send message to the peer endpoint. First one is usable for annotated endpoints only. The user may send the message by returning the message content from the method annotated with `jakarta.websocket.OnMessage`. In the following example the message `m` is sent back to the remote endpoint.

Example 6.2. Sending message in `@OnMessage`

```
1 @OnMessage
2 public String echo(String m) {
3     return m;
4 }
```

The other option how to send a message is to obtain the `jakarta.websocket.RemoteEndpoint` instance via the `jakarta.websocket.Session` instance. See the following example:

Example 6.3. Sending message via `RemoteEndpoint.Basic` instance

```
1 @OnMessage
2 public void echo(String message, Session session) {
3     session.getBasicRemote().sendText(message);
4 }
```

6.4. RemoteEndpoint

The interface `jakarta.websocket.RemoteEndpoint`, part of Java API for WebSocket, is designed to represent the other end of the communication (related to the endpoint), so the developer uses it to send the message. There are two basic interfaces the user may use - `jakarta.websocket.RemoteEndpoint$Basic` and `jakarta.websocket.RemoteEndpoint$Async`.

6.4.1. `jakarta.websocket.RemoteEndpoint.Basic`

This representation of the peer of a web socket conversation is used to send synchronous messages. The point of completion of the send is defined when all the supplied data

has been written to the underlying connection. The methods for sending messages on the `jakarta.websocket.RemoteEndpoint$Basic` block until this point of completion is reached, except for `jakarta.websocket.RemoteEndpoint$Basic#getSendStream()` and `jakarta.websocket.RemoteEndpoint$Basic#getSendWriter()` which present traditional blocking I/O streams to write messages. See the example "Sending message via `RemoteEndpoint.Basic` instance" to see how the whole text message is send. The following example demonstrates a method which sends the partial text method to the peer:

Example 6.4. Method for sending partial text message

```
1 public void sendPartialTextMessage(String message, Boolean isLast, Session ses
2     try {
3         session.getBasicRemote().sendText(message, isLast);
4     } catch (IOException e) {
5         e.printStackTrace();
6     }
7 }
```

6.4.2. `jakarta.websocket.RemoteEndpoint.Async`

This representation of the peer of a web socket conversation has the ability to send messages asynchronously. The point of completion of the send is defined when all the supplied data has been written to the underlying connection. The completion handlers for the asynchronous methods are always called with a different thread from that which initiated the send.

Example 6.5. Sending mesage the async way using Future

```
1 public void sendWholeAsyncMessage(String message, Session session){
2     Future<Void> future = session.getAsyncRemote().sendText(message);
3 }
```

Chapter 7. Injection Support

As required in Java API for WebSocket, Tyrus supports full field, method and constructor injection using `jakarta.inject.Inject` into all websocket endpoint classes as well as the use of the interceptors on these classes. Except this, Tyrus also supports some of the EJB annotations. Currently `jakarta.ejb.Stateful`, `jakarta.ejb.Singleton` and `jakarta.ejb.Stateless` annotations are supported.

7.1. `jakarta.inject.Inject` sample

The following example presents how to inject a bean to the `jakarta.websocket.server.ServerEndpoint` annotated class using `jakarta.inject.Inject`. Class `InjectedSimpleBean` gets injected into class `SimpleEndpoint` on line 15.

Example 7.1. Injecting bean into `jakarta.websocket.server.ServerEndpoint`

```
1 public class InjectedSimpleBean {
2
3     private static final String TEXT = " (from your server)";
4
5     public String getText() {
6         return TEXT;
7     }
8 }
9
10 @ServerEndpoint(value = "/simple")
11 public class SimpleEndpoint {
12
13     private boolean postConstructCalled = false;
14
15     @Inject
16     InjectedSimpleBean bean;
17
18     @OnMessage
19     public String echo(String message) {
20         return String.format("%s%s", message, bean.getText());
21     }
22 }
```

7.2. EJB sample

The following sample presents how to turn `jakarta.websocket.server.ServerEndpoint` annotated class into `jakarta.ejb.Singleton` and use interceptor on it.

Example 7.2. Echo sample server endpoint.

```
1 @ServerEndpoint(value = "/singleton")
2 @Singleton
3 @Interceptors(LoggingInterceptor.class)
```

```
4 public class SingletonEndpoint {
5
6     int counter = 0;
7     public static boolean interceptorCalled = false;
8
9     @OnMessage
10    public String echo(String message) {
11        return interceptorCalled ? String.format("%s%s", message, counter++) :
12    }
13 }
14
15 public class LoggingInterceptor {
16
17     @AroundInvoke
18    public Object manageTransaction(InvocationContext ctx) throws Exception {
19        SingletonEndpoint.interceptorCalled = true;
20        Logger.getLogger(getClass().getName()).info("LOGGING.");
21        return ctx.proceed();
22    }
23 }
24
```

Chapter 8. Tyrus proprietary configuration

Following settings do have influence on Tyrus behaviour and are *NOT* part of WebSocket specification. If you are using following configurable options, your application might not be easily transferable to other WebSocket API implementation.

8.1. Client-side SSL configuration

When accessing "wss" URLs, Tyrus client will pick up whatever keystore and truststore is actually set for current JVM instance, but that might not be always convenient. WebSocket API does not have this feature (yet, see WEBSOCKET_SPEC-210 [https://java.net/jira/browse/WEBSOCKET_SPEC-210]), so Tyrus exposed two SSL configuration classes `SSLContextConfigurator` [<https://grizzly.java.net/docs/2.3/apidocs/org/glassfish/grizzly/ssl/SSLContextConfigurator.html>] and `SslContextConfigurator` [<https://tyrus.java.net/apidocs/2.1.6/org/glassfish/tyrus/client/SslContextConfigurator.html>], which can be used for specifying all SSL parameters to be used with current client instance. The former configuration class belongs to Grizzly configuration API and therefore works only with Grizzly client. The latter configuration class works with both Grizzly and JDK client and offers some extensions over the Grizzly `SSLContextConfigurator` allowing more control of host verification during the SSL handshake. For more details please refer to the following subsection dealing with host verification. Additionally, WebSocket API does not have anything like a client, only `WebSocketContainer` and it does not have any properties, so you need to use Tyrus specific class - `ClientManager` [<https://tyrus.java.net/apidocs/2.1.6/org/glassfish/tyrus/client/ClientManager.html>].

```
1 final ClientManager client = ClientManager.createClient();
2
3 System.getProperties().put("javax.net.debug", "all");
4 System.getProperties().put(SSLContextConfigurator.KEY_STORE_FILE, "...");
5 System.getProperties().put(SSLContextConfigurator.TRUST_STORE_FILE, "...");
6 System.getProperties().put(SSLContextConfigurator.KEY_STORE_PASSWORD, "...");
7 System.getProperties().put(SSLContextConfigurator.TRUST_STORE_PASSWORD, "...");
8 final SSLContextConfigurator defaultConfig = new SSLContextConfigurator();
9
10 defaultConfig.retrieve(System.getProperties());
11 // or setup SSLContextConfigurator using its API.
12
13 SslContextConfigurator sslContextConfigurator =
14     new SslContextConfigurator(defaultConfig, true, false, false);
15 client.getProperties().put(ClientProperties.SSL_CONTEXT_CONFIGURATOR,
16     sslContextConfigurator);
17 client.connectToServer(... , ClientEndpointConfig.Builder.create().build(),
18     new URI("wss://localhost:8181/sample-echo/echo"));
19 }
```

If there seems to be a problem with Tyrus SSL connection, it is strongly recommended to use - `Djavax.net.debug=all` system property as it provides invaluable information for troubleshooting.

8.1.1. Host verification

One of the key steps when establishing SSL connections is verifying that the host on the certificate sent by the server matches the host Tyrus client tries to connect to and thus preventing a possibility of a man-in-the-

middle attack. Host verification is turned on by default in Tyrus, which means that Tyrus will automatically check that the host provided in the URI in

```
1 client.connectToServer(... , new URI("wss://target-server:8181/application/end  
2
```

matches exactly the host the certificate has been issued for. Exact match is the key word in the previous sentence as host can be either hostname or IP address and those two cannot be used interchangeably. For instance when a certificate has been issued for "localhost", establishing an SSL connection to "wss://127.0.0.1:8181/application/endpoint" will fail as the host does not match the one in the certificate.

The default host verification can be too restrictive for some cases and therefore Tyrus provides users with means to either disable the host verification (highly unrecommended in production) or to implement their own host verifier. Providing custom host verifier will disable the default one. It is also important to note that Grizzly specific `SSLEngineConfigurator` [<https://grizzly.java.net/docs/2.3/apidocs/org/glassfish/grizzly/ssl/SSLEngineConfigurator.html>] does not provide these options and for modifying the default host name verification policy `SslEngineConfigurator` [<https://tyrus.java.net/apidocs/2.1.6/org/glassfish/tyrus/client/SslEngineConfigurator.html>] must be used instead. The following sample shows how to disable host name verification:

```
1 SslEngineConfigurator sslEngineConfigurator = new SslEngineConfigurator(new Ss  
2 sslEngineConfigurator.setHostVerificationEnabled(false)  
3 client.getProperties().put(ClientProperties.SSL_ENGINE_CONFIGURATOR, sslEngine  
4
```

The following sample shows how to register a custom host verifier:

```
1 SslEngineConfigurator sslEngineConfigurator = new SslEngineConfigurator(new Ss  
2 sslEngineConfigurator.setHostnameVerifier(new HostnameVerifier() {  
3     @Override  
4     public boolean verify(String host, SSLSession sslSession) {  
5         Certificate certificate = sslSession.getPeerCertificates()[0];  
6         // validate the host in the certificate  
7     }  
8 });  
9 client.getProperties().put(ClientProperties.SSL_ENGINE_CONFIGURATOR, sslEngine  
10
```

8.2. Asynchronous connectToServer methods

`WebSocketContainer.connectToServer(...)` methods are by definition blocking - declared exceptions needs to be thrown after connection attempt is made and it returns `Session` instance, which needs to be ready for sending messages and invoking other methods, which require already established connection.

Existing `connectToServer` methods are fine for lots of uses, but it might cause issue when you are designing application with highly responsible user interface. Tyrus introduces asynchronous variants to each `connectToServer` method (prefixed with "async"), which returns `Future<Session>`. These methods do only simple check for provided URL and the rest is executed in separate thread. All exceptions thrown during this phase are reported as cause of `ExecutionException` thrown when calling `Future<Session>.get()`.

Asynchronous connect methods are declared on Tyrus implementation of `WebSocketContainer` called `ClientManager`.

```
1 ClientManager client = ClientManager.createClient();
```

```
2 final Future<Session> future = client.asyncConnectToServer(ClientEndpoint.cl
3 try {
4     future.get();
5 } catch (...) {
6 }
```

ClientManager contains async alternative to each connectToServer method.

8.3. Optimized broadcast

One of the typical usecases we've seen so far for WebSocket server-side endpoints is broadcasting messages to all connected clients, something like:

```
1 @OnMessage
2 public void onMessage(Session session, String message) throws IOException {
3     for (Session s : session.getOpenSessions()) {
4         s.getBasicRemote().sendText(message);
5     }
6 }
```

Executing this code might cause serious load increase on your application server. Tyrus provides optimized broadcast implementation, which takes advantage of the fact, that we are sending exactly same message to all clients, so dataframe can be created and serialized only once. Furthermore, Tyrus can iterate over set of opened connections faster than Session.getOpenSession().

```
1 @OnMessage
2 public void onMessage(Session session, String message) {
3     ((TyrusSession) session).broadcast(message);
4 }
```

Unfortunately, WebSocket API forbids anything else than Session in @OnMessage annotated method parameter, so you cannot use TyrusSession there directly and you might need to perform instanceof check.

8.4. Incoming buffer size

Servlet container buffers incoming WebSocket frames and there must be a size limit to precede OutOfMemory Exception and potentially DDoS attacks.

Configuration property is named "org.glassfish.tyrus.servlet.incoming-buffer-size" and you can set it in web.xml (this particular snippet sets the buffer size to 17000000 bytes (~16M payload)):

```
1 <web-app version="2.5" xmlns="http://java.sun.com/xml/ns/javaee" xmlns:xsi="ht
2     xsi:schemaLocation="http://java.sun.com/xml/ns/javaee http://java.sun.com/xm
3
4     <context-param>
5         <param-name>org.glassfish.tyrus.servlet.incoming-buffer-size</param-name>
6         <param-value>17000000</param-value>
7     </context-param>
8 </web-app>
```

Default value is 4194315, which correspond to 4M plus few bytes to frame headers, so you should be able to receive up to 4M long message without the need to care about this property.

Same issue is present on client side. There you can set this property via ClientManager:

```
1 ClientManager client = ClientManager.createClient();
2 client.getProperties().put("org.glassfish.tyrus.incomingBufferSize", 6000000);
3 client.connectToServer( ... )
```

8.5. Shared client container

By default, WebSocket client implementation in Tyrus re-creates client runtime whenever `WebSocketContainer#connectToServer` is invoked. This approach gives us some perks like out-of-the-box isolation and relatively low thread count (currently we have 1 selector thread and 2 worker threads). Also it gives you the ability to stop the client runtime – one Session instance is tied to exactly one client runtime, so we can stop it when Session is closed. This seems as a good solution for most of WebSocket client use cases – you usually use java client from application which uses it for communicating with server side and you typically don't need more than 10 instances (my personal estimate is that more than 90% applications won't use more than 1 connection). There are several reasons for it – of it is just a client, it needs to preserve server resources – one WebSocket connection means one TCP connection and we don't really want clients to consume more than needed. Previous statement may be invalidated by WebSocket multiplexing extension, but for now, it is still valid.

On the other hand, WebSocket client implementations in some other containers took another (also correct) approach – they share client runtime for creating all client connections. That means they might not have this strict one session one runtime policy, they cannot really give user way how he to control system resources, but surely it has another advantage – it can handle much more opened connections. Thread pools are share among client sessions which may or may not have some unforeseen consequences, but if its implemented correctly, it should outperform Tyrus solution mentioned in previous paragraph in some use cases, like the one mentioned in TYRUS-275 [<https://java.net/jira/browse/TYRUS-275>] - performance tests. Reporter created simple program which used WebSocket API to create clients and connect to remote endpoint and he measured how many clients can he create (or in other words: how many parallel client connections can be created; I guess that original test case is to measure possible number of concurrent clients on server side, but that does not really matter for this post). Tyrus implementation loose compared to some other and it was exactly because it did not have shared client runtime capability.

How can you use this feature?

```
1 ClientManager client = ClientManager.createClient();
2
3 client.getProperties().put(ClientProperties.SHARED_CONTAINER, true);
```

You might also want to specify container idle timeout:

```
1 client.getProperties().put(ClientProperties.SHARED_CONTAINER_IDLE_TIMEOUT, 5);
```

Last but not least, you might want to specify thread pool sizes used by shared container (please use this feature only when you do know what are you doing. Grizzly by default does not limit max number of used threads, so if you do that, please make sure thread pool size fits your purpose). Even though the default unlimited thread pool size is sufficient for the vast majority of client usages, it is also important to note that if the max. thread pool size is not specified and the clients which share the thread pool receive a large number of messages at the same moment, a new thread can be created for each of the received messages which might demand large amount of system resources and might even lead to a program failure if the required resources are not available. Therefore for particularly busy clients setting the max thread pool size can be only recommended. The following example shows how to set the maximal thread pool size.

```
1 client.getProperties().put(GrizzlyClientProperties.SELECTOR_THREAD_POOL_CONFIG, 10);
2 client.getProperties().put(GrizzlyClientProperties.WORKER_THREAD_POOL_CONFIG, 10);
```

8.5.1. Custom masking key generator

As a security measure, all frames originating on a websocket client have to be masked with a random 4B value, which must be generated for each frame. Moreover to fully comply with the security requirements of RFC 6455, a masking key of a frame must not be predictable from masking keys of previous frames and therefore Tyrus uses `java.security.SecureRandom` as a default masking key generator. While this is perfectly OK for most Tyrus client use cases, usage of `java.security.SecureRandom` might prove to be a performance issue, when the client is used for instance for highly parallel stress testing, as it uses a synchronized singleton as a random entropy provider in its internals.

To overcome the limitations mentioned above, Tyrus allows replacing the default `java.security.SecureRandom` with more scalable masking key generator. Please, be aware that there might be security implications if you decide not to use a cryptographically secure random number generator in production like the one in the following sample. Moreover the supplied random number generator should be also thread safe. The following example shows, how a custom masking key generator can be configured:

```
1 ClientManager client = ClientManager.createClient();
2 client.getProperties().put(ClientProperties.MASKING_KEY_GENERATOR, new Masking
3
4     private final Random random = new Random();
5
6     @Override
7     public int nextInt() {
8         return random.nextInt();
9     }
10 });
```

It is also important to note that the scalability issue connected to the default masking key generator is not limited to the shared container client configuration, but it is discussed in this section as it is assumed that shared container is used for highly parallel clients handling a lot of traffic, where the method of masking key generation starts to matter.

8.6. WebSocket Extensions

Please note that Extensions support is considered to be experimental and any API can be changed anytime. Also, you should ask yourself at least twice whether you don't want to achieve your goal by other means - WebSocket Extension is very powerful and can easily break your application when not used with care or enough expertise.

WebSocket frame used in ExtendedExtension:

```
1 public class Frame {
2
3     public boolean isFin() { .. }
4     public boolean isRsv1() { .. }
5     public boolean isRsv2() { .. }
6     public boolean isRsv3() { .. }
7     public boolean isMask() { .. }
8     public byte getOpcode() { .. }
9     public long getPayloadLength() { .. }
10    public int getMaskingKey() { .. }
11    public byte[] getPayloadData() { .. }
```



```
12     public boolean isControlFrame() { .. }
13
14     public static Builder builder() { .. }
15     public static Builder builder(Frame frame) { .. }
16
17     public final static class Builder {
18
19     public Builder() { .. }
20     public Builder(Frame frame) { .. }
21     public Frame build() { .. }
22     public Builder fin(boolean fin) { .. }
23     public Builder rsv1(boolean rsv1) { .. }
24     public Builder rsv2(boolean rsv2) { .. }
25     public Builder rsv3(boolean rsv3) { .. }
26     public Builder mask(boolean mask) { .. }
27     public Builder opcode(byte opcode) { .. }
28     public Builder payloadLength(long payloadLength) { .. }
29     public Builder maskingKey(int maskingKey) { .. }
30     public Builder payloadData(byte[] payloadData) { .. }
31 }
```

Frame is immutable, so if you want to create new one, you need to create new builder, modify what you want and build it:

```
1 Frame newFrame = Frame.builder(originalFrame).rsv1(true).build();
```

Note that there is only one convenience method: `isControlFrame`. Other information about frame type etc needs to be evaluated directly from opcode, simply because there might not be enough information to get the correct outcome or the information itself would not be very useful. For example: opcode 0x00 means continuation frame, but you don't have any chance to get the information about actual type (text or binary) without intercepting data from previous frames. Consider Frame class as raw representation as possible. `isControlFrame()` can be also gathered from opcode, but it is at least always deterministic and it will be used by most of extension implementations. It is not usual to modify control frames as it might end with half closed connections or unanswered ping messages.

ExtendedExtension representation needs to be able to handle extension parameter negotiation and actual processing of incoming and outgoing frames. It also should be compatible with existing `jakarta.websocket.Extension` class, since we want to re-use existing registration API and be able to return new extension instance included in response from `List<Extension> Session.getNegotiatedExtensions()` call. Consider following:

```
1 public interface ExtendedExtension extends Extension {
2
3     Frame processIncoming(ExtensionContext context, Frame frame);
4     Frame processOutgoing(ExtensionContext context, Frame frame);
5
6     List onExtensionNegotiation(ExtensionContext context, List requestedParameters);
7     void onHandshakeResponse(ExtensionContext context, List responseParameters);
8
9     void destroy(ExtensionContext context);
10
11     interface ExtensionContext {
12
13         Map<String, Object> getProperties();
14     }
15 }
```

15 }

ExtendedExtension is capable of processing frames and influence parameter values during the handshake. Extension is used on both client and server side and since the negotiation is only place where this fact applies, we needed to somehow differentiate these sides. On server side, only onExtensionNegotiation(..) method is invoked and on client side onHandshakeResponse(..). Server side method is a must, client side could be somehow solved by implementing ClientEndpointConfig.Configurator#afterResponse(..) or calling Session.getNegotiatedExtensions(), but it won't be as easy to get this information back to extension instance and even if it was, it won't be very elegant. Also, you might suggest replacing processIncoming and processOutgoing methods by just onprocess(Frame) method. That is also possible, but then you might have to assume current direction from frame instance or somehow from ExtensionContext, which is generally not a bad idea, but it resulted it slightly less readable code.

ExtensionContext and related lifecycle method is there because original jakarta.websocket.Extension is singleton and ExtendedExtension must obey this fact. But it does not meet some requirements we stated previously, like per connection parameter negotiation and of course processing itself will most likely have some connection state. Lifecycle of ExtensionContext is defined as follows: ExtensionContext instance is created right before onExtensionNegotiation (server side) or onHandshakeResponse (client side) and destroyed after destroy method invocation. Obviously, processIncoming or processOutgoing cannot be called before ExtensionContext is created or after is destroyed. You can think of handshake related methods as @OnOpen and destroy as @OnClose.

For those more familiar with WebSocket protocol: process*(ExtensionContext, Frame) is always invoked with unmasked frame, you don't need to care about it. On the other side, payload is as it was received from the wire, before any validation (UTF-8 check for text messages). This fact is particularly important when you are modifying text message content, you need to make sure it is properly encoded in relation to other messages, because encoding/decoding process is stateful – remainder after UTF-8 coding is used as input to coding process for next message. If you want just test this feature and save yourself some headaches, don't modify text message content or try binary messages instead.

8.6.1. ExtendedExtension sample

Let's say we want to create extension which will encrypt and decrypt first byte of every binary message. Assume we have a key (one byte) and our symmetrical cipher will be XOR. (Just for simplicity (a XOR key XOR key) = a, so encrypt() and decrypt() functions are the same).

```
1 public class CryptoExtension implements ExtendedExtension {
2
3     @Override
4     public Frame processIncoming(ExtensionContext context, Frame frame) {
5         return lameCrypt(context, frame);
6     }
7
8     @Override
9     public Frame processOutgoing(ExtensionContext context, Frame frame) {
10        return lameCrypt(context, frame);
11    }
12
13    private Frame lameCrypt(ExtensionContext context, Frame frame) {
14        if(!frame.isControlFrame() && (frame.getOpcode() == 0x02)) {
15            final byte[] payloadData = frame.getPayloadData();
16            payloadData[0] ^= (Byte)(context.getProperties().get("key"));
17
18            return Frame.builder(frame).payloadData(payloadData).build();
```

```
19         } else {
20             return frame;
21         }
22     }
23
24     @Override
25     public List onExtensionNegotiation(ExtensionContext context,
26                                     List requestedParameters) {
27         init(context);
28         // no params.
29         return null;
30     }
31
32     @Override
33     public void onHandshakeResponse(ExtensionContext context,
34     List responseParameters) {
35         init(context);
36     }
37
38     private void init(ExtensionContext context) {
39         context.getProperties().put("key", (byte)0x55);
40     }
41
42     @Override
43     public void destroy(ExtensionContext context) {
44         context.getProperties().clear();
45     }
46
47     @Override
48     public String getName() {
49         return "lame-crypto-extension";
50     }
51
52     @Override
53     public List getParameters() {
54         // no params.
55         return null;
56     }
57 }
```

You can see that ExtendedExtension is slightly more complicated than original Extension so the implementation has to be also not as straightforward.. on the other hand, it does something. Sample code above shows possible simplification mentioned earlier (one process method will be enough), but please take this as just sample implementation. Real world case is usually more complicated.

Now when we have our CryptoExtension implemented, we want to use it. There is nothing new compared to standard WebSocket Java API, feel free to skip this part if you are already familiar with it. Only programmatic version will be demonstrated. It is possible to do it for annotated version as well, but it is little bit more complicated on the server side and I want to keep the code as compact as possible.

Client registration

```
1 ArrayList extensions = new ArrayList();
2 extensions.add(new CryptoExtension());
3
```

```
4 final ClientEndpointConfig clientConfiguration =
5     ClientEndpointConfig.Builder.create()
6     .extensions(extensions).build();
7
8 WebSocketContainer client = ContainerProvider.getWebSocketContainer();
9 final Session session = client.connectToServer(new Endpoint() {
10     @Override
11     public void onOpen(Session session, EndpointConfig config) {
12         // ...
13     }
14 }, clientConfiguration, URI.create(/* ... */));
```

Server registration:

```
1 public class CryptoExtensionApplicationConfig implements ServerApplicationConf
2
3     @Override
4     public Set getEndpointConfigs(Set<Class<? extends Endpoint>> endpointClass
5         Set endpointConfigs = new HashSet();
6         endpointConfigs.add(
7             ServerEndpointConfig.Builder.create(EchoEndpoint.class, "/echo")
8             .extensions(Arrays.asList(new CryptoExtension()))).build()
9         );
10     return endpointConfigs;
11 }
12
13 @Override
14 public Set<Class<?>> getAnnotatedEndpointClasses(Set<Class<?>> scanned) {
15     // all scanned endpoints will be used.
16     return scanned;
17 }
18 }
19
20 public class EchoEndpoint extends Endpoint {
21     @Override
22     public void onOpen(Session session, EndpointConfig config) {
23         // ...
24     }
25 }
```

CryptoExtensionApplicationConfig will be found by servlets scanning mechanism and automatically used for application configuration, no need to add anything (or even have) web.xml.

8.6.2. Per Message Deflate Extension

The original goal of whole extension support was to implement Permessage extension as defined in draft-ietf-hybi-permessage-compression-15 and we were able to achieve that goal. Well, not completely, current implementation ignores parameters. But it seems like it does not matter much, it was tested with Chrome and it works fine. Also it passes newest version of Autobahn test suite, which includes tests for this extension.

see PerMessageDeflateExtension.java (compatible with draft-ietf-hybi-permessage-compression-15, autobahn test suite) and XWebKitDeflateExtension.java (compatible with Chrome and Firefox – same as previous, just different extension name)

8.7. Client reconnect

If you need semi-persistent client connection, you can always implement some reconnect logic by yourself, but Tyrus Client offers useful feature which should be much easier to use. See short sample code:

```
1 ClientManager client = ClientManager.createClient();
2 ClientManager.ReconnectHandler reconnectHandler = new ClientManager.ReconnectH
3
4     private int counter = 0;
5
6     @Override
7     public boolean onDisconnect(CloseReason closeReason) {
8         counter++;
9         if (counter <= 3) {
10             System.out.println("### Reconnecting... (reconnect count: " + counter +
11             return true;
12         } else {
13             return false;
14         }
15     }
16
17     @Override
18     public boolean onConnectFailure(Exception exception) {
19         counter++;
20         if (counter <= 3) {
21             System.out.println("### Reconnecting... (reconnect count: " + counter +
22
23             // Thread.sleep(...) or something other "sleep-like" expression can be p
24             // to do it here to avoid potential DDos when you don't limit number of
25             return true;
26         } else {
27             return false;
28         }
29     }
30
31     @Override
32     public long getDelay() {
33         return 1;
34     }
35 };
36
37 client.getProperties().put(ClientProperties.RECONNECT_HANDLER, reconnectHandle
38
39 client.connectToServer(...)
```

ReconnectHandler contains three methods, `onDisconnect`, `onConnectFailure` and `getDelay`. First will be executed whenever `@OnClose` annotated method (or `Endpoint.onClose(..)`) is executed on client side - this should happen when established connection is lost for any reason. You can find the reason in methods parameter. Other one, called `onConnectFailure` is invoked when client fails to connect to remote endpoint, for example due to temporary network issue or current high server load. Method `getDelay` is called after any of previous methods returns `true` and the returned value will be used to determine delay before next connection attempt. Default value is 5 seconds.

8.8. Client behind proxy

Tyrus client supports traversing proxies, but it is Tyrus specific feature and its configuration is shown in the following code sample:

```
1
2 ClientManager client = ClientManager.createClient();
3 client.getProperties().put(ClientProperties.PROXY_URI, "http://my.proxy.com:80
4
```

Value is expected to be proxy URI. Protocol part is currently ignored, but must be present.

8.9. JDK 7 client

As has been said in previous chapters both Tyrus client and server were implemented on top of Grizzly NIO framework. This still remains true, but an alternative Tyrus Websocket client implementation based on Java 7 Asynchronous Channel API has been available since version 1.6. There are two options how to switch between client implementations. If you do not mind using Tyrus specific API, the most straightforward way is to use:

```
1
2 final ClientManager client = ClientManager.createClient(JdkClientContainer.cla
3
```

You just have to make sure that the dependency on JDK client is included in your project:

```
<dependency>
  <groupId>org.glassfish.tyrus</groupId>
  <artifactId>tyrus-container-jdk-client</artifactId>
  <version>2.1.6</version>
</dependency>
```

Grizzly client is the default option, so creating a client without any parameters will result in Grizzly client being used.

There is also an option how to use JDK client with the standard Websocket API.

```
1
2 final WebSocketContainer client = ContainerProvider.getWebSocketContainer();
3
```

The code listed above will scan class path for Websocket client implementations. A slight problem with this approach is that if there is more than one client on the classpath, the first one discovered will be used. Therefore if you intend to use JDK client with the standard API, you have to make sure that there is not a Grizzly client on the classpath as it might be used instead.

The main reason why JDK client has been implemented is that it does not have any extra dependencies except JDK 7 and of course some other Tyrus modules, which makes it considerable more lightweight compared to Tyrus Grizzly client, which requires 1.4 MB of dependencies.

It is also important to note that the JDK client has been implemented in a way similar to Grizzly client shared container option, which means that there is one thread pool shared among all clients.

Proxy configuration for JDK client is the same as for Grizzly client shown above.

8.9.1. SSL configuration

Alike in case of Grizzly client, accessing "wss" URLs will cause Tyrus client to pick up whatever keystore and trust store is actually set for the current JVM instance. However, specifying SSL parameters to be used with JDK client instance is little different from Grizzly client, because Grizzly client supports both `SSLEngineConfigurator` [<https://grizzly.java.net/docs/2.3/apidocs/org/glassfish/grizzly/ssl/SSLEngineConfigurator.html>] and `SSLContextConfigurator` [<https://grizzly.java.net/docs/2.3/apidocs/org/glassfish/grizzly/ssl/SSLContextConfigurator.html>] from Grizzly project and `SslEngineConfigurator` [<https://tyrus.java.net/apidocs/2.1.6/org/glassfish/tyrus/client/SslEngineConfigurator.html>] and `SslContextConfigurator` [<https://tyrus.java.net/apidocs/2.1.6/org/glassfish/tyrus/client/SslContextConfigurator.html>] from Tyrus project, but JDK client supports only the Tyrus version of these classes. The following code sample shows an example of some SSL parameters configuration for the JDK client:

```
1
2 SslContextConfigurator sslContextConfigurator = new SslContextConfigurator();
3 sslContextConfigurator.setTrustStoreFile("...");
4 sslContextConfigurator.setTrustStorePassword("...");
5 sslContextConfigurator.setTrustStoreType("...");
6 sslContextConfigurator.setKeyStoreFile("...");
7 sslContextConfigurator.setKeyStorePassword("...");
8 sslContextConfigurator.setKeyStoreType("...");
9 SslEngineConfigurator sslEngineConfigurator = new SslEngineConfigurator(sslCon
10
11 client.getProperties().put(ClientProperties.SSL_ENGINE_CONFIGURATOR, sslEngine
12
```

8.10. Tracing support

Apart from logging, Tyrus supports another useful means for debugging and diagnosing a deployed application which will be referred to as tracing on the following lines. Tracing consists of including vital information about handshake handling into a handshake response. The provided information includes among other things an insight into how Tyrus matches handshake request URI against the URI of the deployed endpoints and how the best matching endpoint is selected. The tracing information is included in a handshake response as a content of HTTP headers with `X-Tyrus-Tracing-` as the header names prefix. All the tracing information will also be available in the server log if the appropriate logging level is set. If it is still unclear, how Tyrus tracing works, please refer to the subsection with title Tracing Examples.

8.10.1. Configuration

Tracing support is disabled by default. You can enable it either "globally" for all application handshake requests or selectively per handshake request. The tracing support activation is controlled by setting the `org.glassfish.tyrus.server.tracingType` configuration property. The property value is expected to be one of the following:

- OFF - tracing support is disabled (default value).
- ON_DEMAND - tracing support is in a stand-by mode; it is enabled selectively per handshake, via a special `X-Tyrus-Tracing-Accept` HTTP header in a handshake request.
- ALL - tracing support is enabled for all handshake requests.

The level of detail of the information provided by Tyrus tracing facility - the tracing threshold - can be customized. The tracing threshold can be set at the application level via `org.glassfish.tyrus.server.tracingThreshold` application configuration property in both Glassfish and Grizzly as will be shown in the following samples, or at a request level, via X-Tyrus-Tracing-Threshold HTTP header in a handshake request. The request-level configuration overrides any application level setting. There are 2 supported levels of detail for Tyrus tracing:

- SUMMARY - very basic summary information about handshake processing
- TRACE - detailed information about handshake processing (default threshold value).

8.10.1.1. Global configuration examples

As has been already said, tracing is disabled by default. The following code sample shows, how ON_DEMAND tracing with level set to SUMMARY can be enabled on Grizzly server:

```
1
2                                     serverProperties.put(TyrusWebSocketEngine.TRACING_
3                                     serverProperties.put(TyrusWebSocketEngine.TRACING_
4
```

Similarly ALL tracing with level set to TRACE (the default) can be enabled on Glassfish server in web.xml:

```
1 <web-app version="2.5" xmlns="http://java.sun.com/xml/ns/javaee" xmlns:xsi="ht
2                                     xsi:schemaLocation="http://java.sun.com/xml/ns/javaee http://j
3
4     <context-param>
5         <param-name>org.glassfish.tyrus.server.tracingType</param-name>
6         <param-value>ALL</param-value>
7     </context-param>
8 </web-app>
```

It has been also already mentioned that the tracing threshold configured on application level can be overridden per handshake request as will be shown in the following section.

8.10.1.2. Configuring tracing via handshake request headers

Whenever the tracing support is active (ON_DEMAND or ALL) you can customize the tracing behaviour by including one or more of the following request HTTP headers in the individual handshake requests:

- X-Tyrus-Tracing-Accept - used to enable the tracing support for the particular request. It is applied only when the application-level tracing support is configured to ON_DEMAND mode. The value of the header is not used by the Tyrus tracing facility and as such it can be any arbitrary (even empty) string.
- X-Tyrus-Tracing-Threshold - used to override the tracing threshold. Allowed values are: SUMMARY, TRACE.

8.10.2. Tracing Examples

An example of a handshake request to a server in ON_DEMAND tracing mode requesting SUMMARY tracing information:

```
1 GET /endpoint/b HTTP/1.1
```



```

2 Connection: Upgrade
3 Host: localhost:8025
4 Origin: localhost:8025
5 Sec-WebSocket-Key: YrFldD8nhRW+6hJ2K/TMqw==
6 Sec-WebSocket-Version: 13
7 Upgrade: websocket
8 X-Tyrus-Tracing-Accept: Whatever
9 X-Tyrus-Tracing-Threshold: SUMMARY

```

An example of a possible response to the request above:

```

1 HTTP/1.1 404 Not found
2 x-tyrus-tracing-00 : [0 ms] Matching request URI /samples-debug/endpoint/b aga
3 x-tyrus-tracing-01 : [0 ms] URIs /samples-debug/endpoint/b and /samples-debug/
4 x-tyrus-tracing-02 : [0 ms] Matching request URI /samples-debug/endpoint/b aga
5 x-tyrus-tracing-03 : [0 ms] URIs /samples-debug/endpoint/b and /samples-debug/
6 x-tyrus-tracing-04 : [0 ms] Matching request URI /samples-debug/endpoint/b aga
7 x-tyrus-tracing-05 : [1 ms] URIs /samples-debug/endpoint/b and /samples-debug/
8 x-tyrus-tracing-06 : [1 ms] Matching request URI /samples-debug/endpoint/b aga
9 x-tyrus-tracing-07 : [1 ms] URIs /samples-debug/endpoint/b and /samples-debug/
10 x-tyrus-tracing-08 : [1 ms] Matching request URI /samples-debug/endpoint/b aga
11 x-tyrus-tracing-09 : [1 ms] Segment "a" does not match
12 x-tyrus-tracing-10 : [1 ms] Matching request URI /samples-debug/endpoint/b aga
13 x-tyrus-tracing-11 : [1 ms] URIs /samples-debug/endpoint/b and /samples-debug/
14 x-tyrus-tracing-12 : [3 ms] Endpoints matched to the request URI: []

```

The time in the square brackets in the sample above is the time since the handshake request has been received.

An example of a possible handshake response from a server in ALL tracing mode with tracing threshold set to TRACE:

```

1 HTTP/1.1 101
2 connection: Upgrade
3 sec-websocket-accept: C8/QbF4Mx9sX31sihUcnI19yqto=
4 upgrade: websocket
5 x-tyrus-tracing-00 : [0 ms] Matching request URI /samples-debug/endpoint/a/b a
6 x-tyrus-tracing-01 : [0 ms] Matching request URI /samples-debug/endpoint/a/b a
7 x-tyrus-tracing-02 : [0 ms] Matching request URI /samples-debug/endpoint/a/b a
8 x-tyrus-tracing-03 : [1 ms] Matching request URI /samples-debug/endpoint/a/b a
9 x-tyrus-tracing-04 : [1 ms] Segment "a" does not match
10 x-tyrus-tracing-05 : [1 ms] Matching request URI /samples-debug/endpoint/a/b a
11 x-tyrus-tracing-06 : [1 ms] URIs /samples-debug/endpoint/a/b and /samples-debu
12 x-tyrus-tracing-07 : [1 ms] Matching request URI /samples-debug/endpoint/a/b a
13 x-tyrus-tracing-08 : [3 ms] Choosing better match from /samples-debug/endpoint
14 x-tyrus-tracing-09 : [3 ms] /samples-debug/endpoint/a/b is an exact match
15 x-tyrus-tracing-10 : [3 ms] Choosing better match from /samples-debug/endpoint
16 x-tyrus-tracing-11 : [3 ms] /samples-debug/endpoint/a/{b} is a better match,
17 x-tyrus-tracing-12 : [3 ms] Choosing better match from /samples-debug/endpoint
18 x-tyrus-tracing-13 : [3 ms] /samples-debug/endpoint/a/{b} is a better match,
19 x-tyrus-tracing-14 : [3 ms] Choosing better match from /samples-debug/endpoint
20 x-tyrus-tracing-15 : [3 ms] /samples-debug/endpoint/a/b is an exact match
21 x-tyrus-tracing-16 : [3 ms] Choosing better match from /samples-debug/endpoint
22 x-tyrus-tracing-17 : [4 ms] /samples-debug/endpoint/a/{b} is a better match,
23 x-tyrus-tracing-18 : [4 ms] Choosing better match from /samples-debug/endpoint

```

```
24 x-tyrus-tracing-19 : [4 ms] /samples-debug/endpoint/{a}/b is a better match,  
25 x-tyrus-tracing-20 : [4 ms] Endpoints matched to the request URI: [/samples-de  
26 x-tyrus-tracing-21 : [4 ms] Endpoint selected as a match to the handshake URI:
```

8.11. Client handshake request and response logging

Tyrus client offers a possibility, how to enable printing of handshake requests and responses to standard output without having to configure Java logging, which is essential when debugging a misbehaving websocket application. This feature is particularly useful with tracing enabled. The following sample shows, how the handshake logging can be enabled:

```
1  
2         ClientManager client = ClientManager.createClient();  
3         client.getProperties().put(ClientProperties.LOG_HTTP_UPGRADE,  
4
```

8.12. JMX Monitoring

Tyrus allows monitoring and accessing some runtime properties and metrics at the server side using JMX (Java management extension technology). The monitoring API has been available since version 1.6 and the following properties are available at runtime through MXBeans. Number of open sessions, maximal number of open session since the start of monitoring and list of deployed endpoint class names and paths are available for each application. Endpoint class name and path the endpoint is registered on, number of open session and maximal number of open sessions are available for each endpoint. Apart from that message as well as error statistics are collected both per application and per individual endpoint.

The following message statistics are monitored for both sent and received messages:

- messages count
- messages count per second
- average message size
- smallest message size
- largest message size

Moreover all of them are collected separately for text, binary and control messages and apart from the statistics being available for the three separate categories, total numbers summing up statistics from the three types of messages are also available.

As has been already mentioned above, Tyrus also monitors errors on both application and endpoint level. An error is identified by the Throwable class name that has been thrown. Statistics are collected about number of times each Throwable has been thrown, so a list of errors together with a number of times each error occurred is available on both application and endpoint level. The monitored errors correspond to invocation of `@OnError` method on an annotated endpoint or its equivalent on a programmatic endpoint (The invocation of `@OnError` method is just an analogy and an error will be monitored even if no `@OnError` method is provided on the endpoint). Errors that occur in `@OnOpen`, `@OnClose` methods and methods handling incoming messages are monitored. Errors that occurred during handshake will not be among the monitored errors.

The collected metrics as well as the endpoint properties mentioned above are accessible at runtime through Tyrus MXBeans. As has been already mention the information is available on both application and endpoint level with each application or endpoint being represented with four MXBeans. One of those MXBeans contains total message statistics for both sent and received messages as well as any properties specific for applications or endpoints such as endpoint path in the case of an endpoint. The other three MXBeans contain information about sent and received text, binary and control messages.

When a user connects to a tyrus application MBean server using an JMX client such as JConsole, they will see the following structure:

- Application 1 - MXBean containing a list of deployed endpoint class names and paths, number of open sessions, maximal number of open sessions, error and total message statistics for the application.
 - message statistics - a directory containing message statistics MXBeans
 - text - MXBean containing text message statistics
 - binary - MXBean containing binary message statistics
 - control - MXBean containing control message statistics
 - endpoints - a directory containing application endpoint MXBeans
 - Endpoint 1 - MXBean containing Endpoint 1 class name and path, number of open sessions, maximal number of open sessions, error and total message statistics for the endpoint.
 - text - MXBean containing text message statistics
 - binary - MXBean containing binary message statistics
 - control - MXBean containing control message statistics
 - Endpoint 2
- Application 2

In fact the monitoring structure described above was a little bit simplistic, because there is an additional monitoring level available, which causes message metrics being also available per session. The monitoring structure is very similar to the one described above, with a small difference that there are four MXBeans registered for each session, which contain text, binary, control and total message statistics. In order to distinguish the two monitoring levels, they will be referred to as endpoint-level monitoring and session-level monitoring.

8.12.1. Configuration

As has been already mentioned, monitoring is supported only on the server side and is disabled by default. The following code sample shows, how endpoint-level monitoring can be enabled on Grizzly server:

```
1
2 serverProperties.put(ApplicationEventListener.APPLICATION_EVENT_LISTENER, new
3
```

Similarly endpoint-level monitoring can be enabled on Grizzly server in the following way:

```
1
2 serverProperties.put(ApplicationEventListener.APPLICATION_EVENT_LISTENER, new
3
```

Monitoring can be configured on Glassfish in web.xml and the following code sample shows endpoint-level configuration:

```
1 <web-app version="2.5" xmlns="http://java.sun.com/xml/ns/javaee" xmlns:xsi="ht
2           xsi:schemaLocation="http://java.sun.com/xml/ns/javaee http
3
4     <context-param>
5         <param-name>org.glassfish.tyrus.core.monitoring.ApplicationEventListen
6         <param-value>org.glassfish.tyrus.ext.monitoring.jmx.SessionlessApplica
7     </context-param>
8 </web-app>
9
```

Similarly session-level monitoring can be configured on Glassfish in web.xml in the following way:

```
1 <web-app version="2.5" xmlns="http://java.sun.com/xml/ns/javaee" xmlns:xsi="ht
2           xsi:schemaLocation="http://java.sun.com/xml/ns/javaee http
3
4     <context-param>
5         <param-name>org.glassfish.tyrus.core.monitoring.ApplicationEventListen
6         <param-value>org.glassfish.tyrus.ext.monitoring.jmx.SessionAwareApplic
7     </context-param>
8 </web-app>
9
```

8.13. Maximal number of open sessions on server-side

Tyrus offers a few ways to limit the number of open sessions, which can be used to save limited resources on a server hosting system. The limits can be configured in several scopes:

- per whole application
- per endpoint
- per remote address (client IP address)

If the number of simultaneously opened sessions exceeds any of these limits, Tyrus will close the session with close code 1013 - Try Again Later.

Limits mentioned above can be combined together. For example, let's say we have an application with two endpoints. Overall limit per application will be 1000 open sessions and the first one, non-critical endpoint, will be limited to 75 open sessions at maximum. So we know that the second endpoint can handle 925-1000 opened sessions, depends on how many open sessions are connected to the first endpoint (0-75).

8.13.1. Maximal number of open sessions per application

This configuration property can be used to limit overall number of open sessions per whole application. The main purpose of this configurable limit is to restrict how many resources the application can consume.

The number of open sessions per whole application can be configured by setting property `org.glassfish.tyrus.maxSessionsPerApp`. Property can be used as `<context-param>` in `web.xml` or as an entry in parameter map in (standalone) Server properties.

Note that only positive integer is allowed.

This example will set maximal number of open sessions per whole application to 500:

```
<web-app version="2.5" xmlns="http://java.sun.com/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://java.sun.com/xml/ns/javaee http://java.sun.com
  <context-param>
    <param-name>org.glassfish.tyrus.maxSessionsPerApp</param-name>
    <param-value>500</param-value>
  </context-param>
</web-app>
```

8.13.2. Maximal number of open sessions per remote address

The number of open sessions per remote address can be configured by setting property `org.glassfish.tyrus.maxSessionsPerRemoteAddr`. Property can be used as `<context-param>` in `web.xml` or as an entry in parameter map in (standalone) Server properties.

Remote address value is obtained from `ServletRequest#getRemoteAddr()` [[http://docs.oracle.com/javaee/6/api/javax/servlet/ServletRequest.html#getRemoteAddr\(\)](http://docs.oracle.com/javaee/6/api/javax/servlet/ServletRequest.html#getRemoteAddr())] or its alternative when using Grizzly server implementation. Beware that this method returns always the last node which sending HTTP request, so all clients behind one proxy will be treated as clients from single remote address.

Note that only positive integer is allowed.

This example will set maximal number of open sessions from unique IP address or last proxy to 5:

```
<web-app version="2.5" xmlns="http://java.sun.com/xml/ns/javaee" xmlns:xsi="http://
  xsi:schemaLocation="http://java.sun.com/xml/ns/javaee http://java.sun.com
  <context-param>
    <param-name>org.glassfish.tyrus.maxSessionsPerRemoteAddr</param-name>
    <param-value>5</param-value>
  </context-param>
</web-app>
```

8.13.3. Maximal number of open sessions per endpoint

Set maximum number of sessions in annotated endpoint:

```
1
2 import jakarta.websocket.OnOpen;
3 import jakarta.websocket.Session;
4 import jakarta.websocket.server.ServerEndpoint;
```

```
5
6 import org.glassfish.tyrus.core.MaxSessions;
7
8 /**
9  * Annotated endpoint.
10 */
11 @MaxSessions(100)
12 @ServerEndpoint(value = "/limited-sessions-endpoint")
13 public static class LimitedSessionsEndpoint {
14     @OnOpen
15     public void onOpen(Session s) {
16         ...
17     }
18     ...
19 }
20
```

Set maximum number of sessions for programmatic endpoint:

```
1
2 TyrusServerEndpointConfig.Builder.create(LimitedSessionsEndpoint.class,
3     "/limited-sessions-endpoint").maxSessions(100).build();
4
```

Note that only positive integer is allowed.

8.14. Client HTTP Authentication

For server endpoints which is protected by HTTP authentication, Tyrus provides a mechanism to authenticate client. When client receives HTTP response status code 401 - `Unauthorized`, then Tyrus extracts required scheme from `WWW-Authenticate` challenge. Then it chooses an authenticator from a map of registered authenticators and uses configured credentials. If no proper authenticator is found or credentials are missing, then `AuthenticationException` is thrown before the handshake can be done. There are implementations of the two most used authentication schemes in Tyrus: BASIC and DIGEST, but it is also possible to implement your own authenticator and register it with a configuration builder `org.glassfish.tyrus.client.auth.AuthConfig.Builder` or even to override default BASIC or DIGEST auth implementations. If no `org.glassfish.tyrus.client.auth.AuthConfig` client property is set, then default configuration is used. It is constructed as you can see below: BASIC and DIGEST internal implementations are enabled by default.

Please note that Basic Authentication scheme should be used over HTTPS connection only.

8.14.1. Credentials

Credentials are required for both implemented authentication schemes in Tyrus. You can pass an instance into `ClientManager` as a property:

```
1
2
3 client.getProperties().put(ClientProperties.CREDENTIALS, new Credentials("ws
4
5
```

8.14.2. Auth Configuration

`org.glassfish.tyrus.client.auth.AuthConfig` provides a way to configure of HTTP authentication schemes. Creating an instance of `org.glassfish.tyrus.client.auth.AuthConfig` is optional. If you don't specify `AuthConfig`, then default instance will be created like in following code listing

```
1
2
3  AuthConfig authConfig = AuthConfig.Builder.create().build();
4  ClientManager client = ClientManager.createClient();
5  client.getProperties().put(ClientProperties.AUTH_CONFIG, authConfig);
6
7
```

If authentication is required after an initial upgrade request, Tyrus chooses a proper authentication scheme based on a received challenge from server. There are two HTTP authentication scheme implemented and registered by default.

8.14.3. User defined authenticator

Tyrus provides an option to implement your own client HTTP authenticator by extending `org.glassfish.tyrus.client.auth.Authenticator` and implementing `generateAuthorizationHeader`. Request URI, `WWW-Authenticate` response header and provided `Credentials` are passed as parameters. Method must return response to authentication challenge as it is required by HTTP server. An instance of the implemented class must be passed to the Tyrus configuration with `org.glassfish.tyrus.client.auth.AuthConfig.Builder#setAuthScheme(String scheme, Authenticator userDefinedAuthenticator)` and created `AuthConfig` instance must be put into client properties.

`Authenticator.java`

```
1
2
3 package org.glassfish.tyrus.client;
4
5 import java.net.URI;
6
7 /**
8  * Http Authentication provider.
9  * Class generates authorization token as a input for {@code Authorization} HT
10 *
11 * @author Ondrej Kosatka (ondrej.kosatka at oracle.com)
12 */
13 public abstract class Authenticator {
14
15     /**
16      * Generates authorization tokens as a input for {@code Authorization} HT
17      * @param uri URI is needed for generating authorization tokens for some a
18      * @param wwwAuthenticateHeader a value of header {@code WWW-Authenticate}
19      * @param credentials credentials.
20      * @return generated {@code String} value of {@code Authorization}.
21      * @throws AuthenticationException if is not possible to create auth token
```

```
22     */
23     public abstract String generateAuthorizationHeader(final URI uri, final St
24
25 }
26
27
```

8.14.4. Examples

The simplest way to setup Tyrus authentication is by adding client property `ClientProperties.CREDENTIALS`

```
1
2
3     client.getProperties().put(ClientProperties.CREDENTIALS, new Credentials("ws
4
5
```

How to configure Tyrus with suppressing Basic authentication, even if server side challenges Basic authentication scheme.

```
1
2
3     AuthConfig authConfig = AuthConfig.Builder.create().
4         disableBasicAuth().
5         build();
6     Credentials credentials = new Credentials("ws_user", "password");
7     client.getProperties().put(ClientProperties.AUTH_CONFIG, authConfig);
8     client.getProperties().put(ClientProperties.CREDENTIALS, credentials);
9
10
```

How to configure Tyrus using user defined DIGEST authentication and Tyrus Basic authentication. User defined authentication provider `MyOwnDigestAuthenticator` must extend `org.glassfish.tyrus.client.auth.Authenticator`.

```
1
2
3     AuthConfig authConfig = AuthConfig.Builder.create().
4         putAuthProvider("Digest", new MyOwnDigestAuthen
5         build();
6     Credentials credentials = new Credentials("ws_user", "password");
7     client.getProperties().put(ClientProperties.AUTH_CONFIG, authConfig);
8     client.getProperties().put(ClientProperties.CREDENTIALS, credentials);
9
10
```

How to configure Tyrus using user defined NTLM authentication and suppress Tyrus Basic authentication, even if server side challenges Basic authentication scheme.. User defined authentication provider `MyOwnNTLMAuthenticator` must extend `org.glassfish.tyrus.client.auth.Authenticator`.

```
1
2
3     AuthConfig authConfig = AuthConfig.Builder.create().
```



```
4             disableBasicAuth().
5             putAuthProvider("NTLM", new MyOwnNTLMAuthentica
6             build();
7 Credentials credentials = new Credentials("ws_user", "password");
8 client.getProperties().put(ClientProperties.AUTH_CONFIG, authConfig);
9 client.getProperties().put(ClientProperties.CREDENTIALS, credentials);
10
11
```

8.15. Client HTTP Redirect

Another Tyrus feature is HTTP redirect. If client received 3xx HTTP Redirect response code during a handshake and HTTP Redirect is allowed (by `ClientProperty.REDIRECT_ENABLED` property) then client engine transparently follows the URI contained in received HTTP response header `Location` and sends upgrade request to the new URI. Redirects can be chained up to limit set in `ClientProperty.REDIRECT_THRESHOLD`, whilst default value is 5. If HTTP redirect failed by any reason, `RedirectException` is thrown.

8.15.1. Supported HTTP response codes

List of 3xx HTTP response codes which can be automatically redirect

- 300 - Multiple Choices
- 301 - Moved permanently
- 302 - Found
- 303 - See Other (since HTTP/1.1)
- 307 - Temporary Redirect (since HTTP/1.1)
- 308 - Permanent Redirect (Experimental RFC; RFC 7238)

8.15.2. Configuration

8.15.2.1. Enabling

For enabling HTTP Redirect feature, `ClientProperty.REDIRECT_ENABLED` must be explicitly set to `true` (default value is `false`), otherwise `RedirectException` will be thrown, when any of supported HTTP Redirect response codes (see above).

```
client.getProperties().put(ClientProperties.REDIRECT_ENABLED,
```

8.15.2.2. Threshold

`ClientProperty.REDIRECT_THRESHOLD` is property which can be used to limit maximal number of chained redirect. Positive integer is expected and default value is 5.

```
client.getProperties().put(ClientProperties.REDIRECT_THRESHOLD
```

8.15.3. Exception handling

`RedirectException` is set as a cause of `DeploymentException` when any of the supported Redirection HTTP response status codes (see above) was received and `WebSocketContainer.connectToServer(...)` fails because of any of the following reasons:

- `ClientProperties.REDIRECT_ENABLED` property is not set to true.
- Value of `ClientProperties.REDIRECT_THRESHOLD` is not assignable to Integer.
- Number of chained redirection exceeds a value of `ClientProperties.REDIRECT_THRESHOLD` (default value is 5).
- Infinite redirection loop is detected.
- Location response header is missing, is empty or does not contain a valid URI.

8.16. Client support for HTTP status 503 - Service Unavailable with Retry-After header

Tyrus offers automatic handling of HTTP status code 503 - Service Unavailable, which can be returned from server when temporarily overloaded or down for maintenance. When `Retry-After` header is included in the response, client will parse the value and schedule another reconnect attempt.

This feature is disabled by default.

The implementation limits connection attempts to 5, each with reconnect delay not bigger than 300 seconds. Other values or conditions can be handled by custom `ReconnectHandler` (see `RetryAfterException`).

8.16.1. Configuration

```
1
2
3      final ClientManager client = ClientManager.createClient();
4      client.getProperties().put(ClientProperties.RETRY_AFTER_SERVICE_UN
5
```